GOSIM

GOSIM Shenzhen 2026

OctoLoop

从单一 Agent 到 Agent 生产线

赵悦



Westlake：让未修改的 Android 原包运行在 OpenHarmony 上

工程背景

从 APK 出发，分析运行阻塞
修复公共能力并验证实际效果

我希望走向的目标

面向更多 Android 应用移植
建设可重复运行与验证的生产线

先介绍这个工程，再看过去我是怎样推进它的

过去，我是怎样推进这个工程的



人组织每次诊断、执行、人工验证，再开始下一项

这种工作方式遇到了什么瓶颈

AI 辅助局部执行，人负责把整个任务推到终点

协调反复发生	重讲背景、拆任务、安排执行顺序
判断集中在人	Westlake 的“上屏”交付，仍需逐张核对截图
经验难以复用	有效方法与失败教训散落在会话里

单个 Agent 更能干，还不足以解决整个过程的组织问题

OctosCode 与 OctoLoop 是什么

OctosCode 是编码工具，OctoLoop 是双环协作机制

OctosCode

基于 Octos 的终端编码助手
用 TUI 交互、看改动和审批
由 Octos runtime 执行 Agent

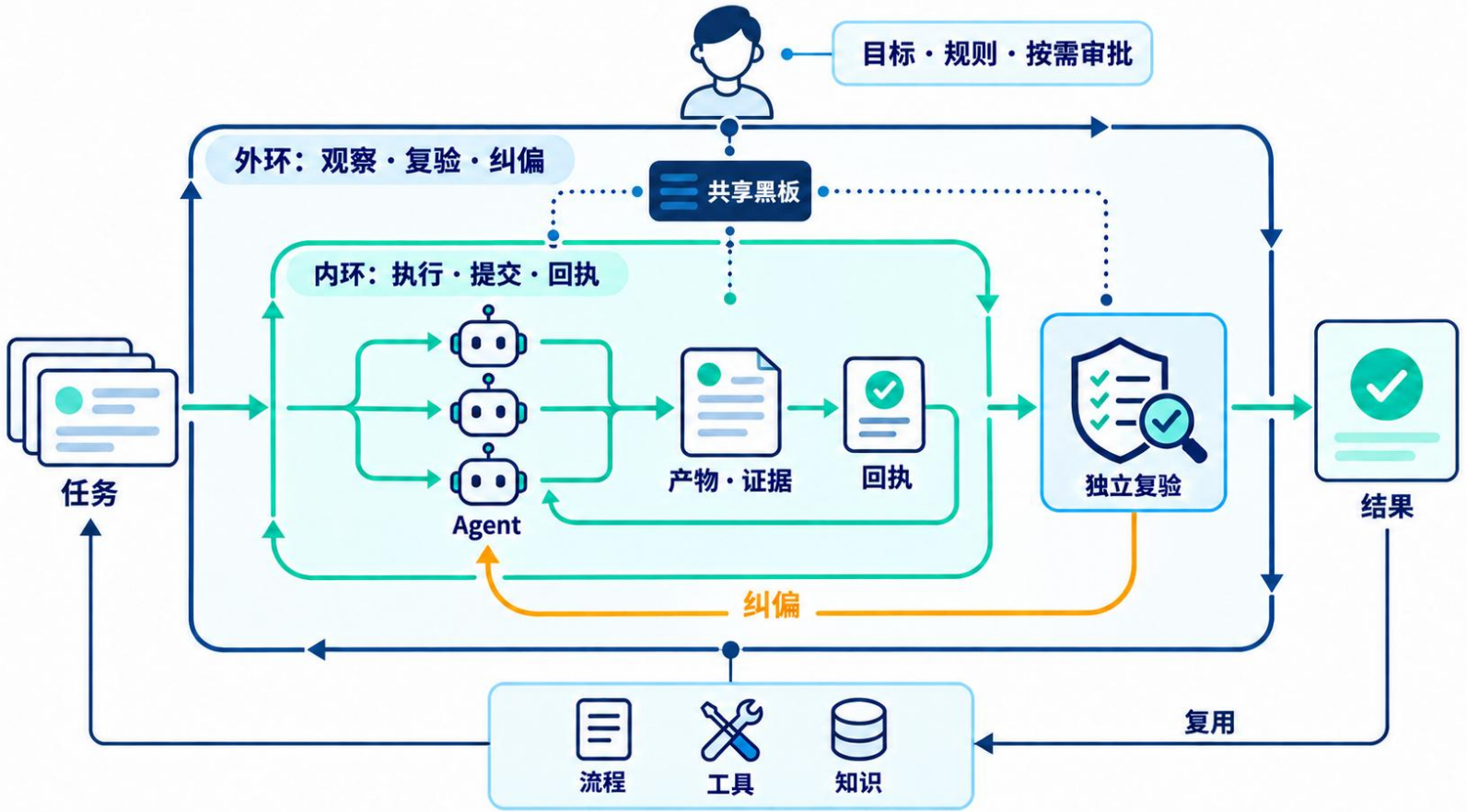
OctoLoop

外环观察、复验与纠偏
内环执行、提交与回执
组织多个 Agent 持续推进

Octoscode 项目｜github.com/octos-org/octoscode

Octos 提供执行底座，OctosCode 提供入口，OctoLoop 组织协作

范式转换：Agent 的“工业革命”



从单一 Agent 到 Agent 生产线

人退到生产线设计与改进的位置：定义目标、规则、验收与审批

双环怎样针对旧问题改进

把持续执行、独立判断和经验复用纳入同一工作过程

重复协调	master 推进目标，peer 按明确任务并行执行
交件难判断	外环独立复验，把偏差变成明确整改
进展难接续	黑板保留指导、回执与证据路径
经验易散失	流程、工具和知识进入下一轮执行

人确定目标与边界，双环在授权范围内推进与纠偏

运行与交互分离，便于嵌入和调度

Octos 管执行，客户端通过 API 控制运行时

各层各司其职

核心运行时管理执行与状态
server 通过 OUP 暴露控制能力
OctosCode TUI 负责交互与审批

支持不同控制入口

应用可嵌入 Octos 核心
调度器通过 API 派工、观察
外部 Agent 经 bridge 发出干预

OUP：JSON-RPC 2.0，支持 stdio 与 WebSocket

控制入口开放，外环就能接入并驱动内环



下达目标与干预

派工、观察、收集结果

执行任务并返回事件

OUP 负责控制运行时；OLP 约定双环如何协调与验收

Peer agents：有明确交付的并行协作

master 持续推进目标，peer 带着任务简报工作

按需要组织分工

分析问题、实现改动
测试与证据收集
结果汇总后交给外环审查

交接包含执行条件

任务背景、范围和验收要求
交付物及证据位置
遇到阻塞时的升级路径

并行工作由目标和交付约束连接，避免各做各的

黑板：双环之间的持久通讯

协作通过可追踪的记录发生，而不依赖共享记忆

外环写指导	编号、问题、验收要求与证据依据
内环写回执	完成、有证据的异议，或明确阻塞
双方核对交付	关联代码、运行结果与整改记录
历史留痕	已处理的指导归档，不重复执行

OLP 约定通讯纪律，黑板连接指导、执行与反馈

执行闭环：外环独立复验与纠偏

内环的完成声明，需要由外环用证据核对

内环执行与交件

提交改动、交付物与回执
声明实际验证过的范围
提供可复现的证据

外环观察与判断

独立复验结果和差异
将不通过的项写成整改
分歧或权限问题升级给人

执行产生结果，复验产生反馈，反馈改变下一步行动

进化环：整体调度、测量与改进

外环把本轮证据转成下一轮的调度与改进依据

整体调度	围绕目标分工，按阻塞与结果调整下一步
测量结果	验收是否通过、验证覆盖到哪里、哪里反复返工
分析原因	区分任务问题、流程缺口和工具缺口
验证改进	改变做法后，再用下一轮证据检验

改流程 → 改工具 → 积累知识 → 推进代码 → 下一轮验证

Westlake：前面的三个问题怎样解决

让派工有交接、交付有复验、修复能跨任务复用

协调反复发生

外环组织分工，内环按交付接续

安装器授权与入口补丁来自不同车道，集成为同一版本

判断集中在人

内环交件，外环复验，不通过就继续整改

桌面交件未通过读图验收，整改后真名与图标才获签认

经验难以复用

公共层修复经多应用验证后冻结复用

同一安装器修复让四个应用显示自身页面，登记 FZ-001

记录证明这些协作与验收实际完成，尚未量化人力节省或总体成功率

后台启动被拒，修复统一安装器后出现了自身页面

OctoLoop 怎样组织解决

不同车道分别修授权与入口
集成为同一版本，按黑板交接
外环核截图与日志，验收后冻结

实际发生的变化

Tusky：白屏变为登录页
K-9 / Thunderbird：进入升级页
File Manager：显示文件列表

历史验证包含服务重启，页面出现不等于功能全部通过

第二块板复现，FZ-001 留存产物哈希、截图与日志

2026-10-08：128 个应用能上屏，多数功能尚未逐项验收

未来方向：结构化调度与更快反馈

展望：把黑板从可读记录，推进为更明确的调度依据

更结构化的调度黑板

任务身份、状态、依赖与责任方
验收条件和证据关联
支持机器检查与一致性核对

更快的反馈响应

缩短交件到复验的等待
更及时地发现阻塞与偏差
探索事件触发与优先级机制

目标：更少重复协调，更及时地把反馈送回执行端

未来方向：持久调度与长流程上下文

展望：让内外环协作跨会话接续，让长黑板保持可用

持久化的内外环调度

持久记录调度状态与交接点
恢复时辨认已完成和待处理项
探索更完整的接续与去重机制

黑板过长的处理方法

活跃任务与历史记录分层
归档、摘要与按需检索
为当前任务选择相关上下文

待探索：如何在压缩上下文时，保留证据、责任与可追踪性

GOSIM

GOSIM Shenzhen 2026

OctoLoop

人定义目标与规则
双环组织 Agent 生产线
测量与进化改进整个系统

Octoscode 项目｜github.com/octos-org/octoscode

赵悦　谢谢

